

大语言模型与 GPT 应用

数据科学与大数据技术专业

第 8 讲

1 导读

除本导读外, 本讲共有 10 个主体节. 它们从 ChatGPT 文本框背后的 token 生成机制讲起, 逐步说明预训练、后训练、幻觉、工具使用和强化学习, 再把这些原理落实到搜索、文件、代码、多模态、记忆和自定义工具等具体使用场景中.

1. **问题起点: ChatGPT 文本框背后是什么:** 说明对话界面为什么应被理解为一个带角色标记的 token 流, 而不是一个真正持续存在的“人”.
2. **预训练数据与 tokenization:** 说明互联网文本如何被筛选成训练语料, 原始文字又如何被切成模型能处理的 token.
3. **预训练目标与 Transformer:** 说明下一个 token 预测如何形成训练目标, 以及 Transformer 为什么适合做大规模并行语言建模.
4. **推理、采样与基础模型:** 说明训练结束后的模型如何逐 token 生成文本, 并区分 base model 与 instruct model.
5. **后训练、助手人格与幻觉:** 说明 SFT 如何把续写器变成助手, 幻觉为什么会自然出现, 以及事实性训练如何缓解这一问题.
6. **工具使用与工作记忆:** 说明搜索、文件、代码和计算器等工具如何把模糊参数记忆转成当前可读的上下文.
7. **强化学习与思考模型:** 说明为什么仅靠模仿示例还不够, 以及 RL、RLHF 和 thinking model 各自解决什么问题.
8. **GPT 应用的任务选择:** 说明什么时候可以直接问模型, 什么时候应使用搜索、深度研究或多模型比较.
9. **文件、代码与编程工作流:** 说明如何让模型阅读具体材料、运行 Python、生成图表, 以及为什么真实编程应把模型放进代码库.
10. **多模态、记忆与自定义工具:** 说明语音、图片、视频、跨对话记忆、自定义指令和自定义 GPT 如何扩展 LLM 的应用边界.

如果细节都忘了: 最应该记住的是, LLM 的核心不是“魔法大脑”, 而是一个在上下文中预测下一个 token 的神经网络. 真正好用的 GPT 产品, 是把模型、对话协议、上下文窗口、搜索、文件、代

码、多模态输入输出和记忆系统组织在一起。因此使用 LLM 时要先判断：这个问题能否靠模型参数中的模糊知识回答，还是必须把材料、来源或工具接进上下文。

问题思考：

- 为什么理解 token、上下文和工具调用，比单纯记住某个产品按钮更重要？

2 问题起点：ChatGPT 文本框背后是什么

本节导读：

- **本节目的：**把 ChatGPT 这类聊天界面还原成模型真正处理的对象，避免把气泡对话误解成一个持续存在的智能体自我。
- **为什么学：**后面讨论预训练、后训练、幻觉、上下文窗口和工具调用时，都要回到“文本被编码成 token 序列”这一基本机制。
- **核心内容：**用户消息、助手消息、系统提示和工具结果最终都会进入同一个一维 token 序列；模型在序列末尾继续生成 token。
- **最小例子：**用户输入 `What is 2 + 2?`，产品层会把它连同 `user/assistant` 角色边界放进上下文，模型从 `assistant` 开始的位置继续生成 `4` 以及后续解释。
- **最该记住：**聊天气泡是界面层，token 流才是模型直接看到的工作材料。

1. 从气泡界面到 token 流

- 用户看到的是一轮轮聊天气泡：用户提问，助手回答，用户追问，助手继续解释。但语言模型本身通常不直接处理“气泡对象”，而是处理一个线性化后的 token 序列。
- 产品层会把每条消息的角色和边界编码进上下文。例如 `system` 消息规定回答风格，`user` 消息提供用户请求，`assistant` 消息记录模型已经回答的内容。这些信息最后都会变成模型可读的符号序列。
- 因此，New Chat 的意义不是换了一个真正的人，而是清空或更换当前上下文窗口。如果旧对话历史不再放入窗口，模型就不能直接读取刚才讨论过的细节。

2. 上下文窗口是工作记忆

- 上下文窗口可以理解为本次推理中模型能直接看到的工作记忆。只要一段材料在窗口里，模型就可以在生成后续 token 时参考它；如果材料不在窗口里，模型只能依赖参数中压缩过的模糊知识。
- 这解释了为什么长对话会变慢、变贵，也可能变得不稳定。窗口越长，模型每一步要参考的信息越多；如果里面混入大量无关历史，当前任务的有效信号就会被稀释。
- 切换主题时新建对话，本质上是在保护工作记忆。对当前任务有用的材料应留下，无关材料应清掉。

3. 模型没有天然的产品自我

- 语言模型每次请求时读取上下文，执行前向计算，生成 token，然后这次计算结束。它不像人一样在不同请求之间天然保持连续的自我意识。

- 当模型回答“我是谁”“谁开发了我”“知识截止日期是什么”时，这些信息通常来自系统消息、后训练数据或产品层注入的说明。如果这些设定缺失或冲突，模型可能根据互联网上常见的助手文本猜测。
- 因此，身份回答应被理解为当前上下文和训练分布共同塑造出的角色设定，不是模型对内部状态的可靠内省。

4. 对话协议会塑造模型行为

- 同一个模型若使用正确的 chat template, 即按它训练时见过的 system、user、assistant 格式组织输入，通常会稳定地表现为助手。如果只把一句问题当作普通文本前缀塞给模型，它可能更像在续写网页或论坛文本。
- 这说明 ChatGPT 式体验不是单靠一个神经网络文件实现的。它依赖模型参数、tokenizer、对话模板、系统提示、采样策略和产品工具共同组成的系统。
- 学会使用 LLM, 就要同时理解模型层和产品层：模型负责在上下文中续写，产品层负责把用户意图、历史消息和工具结果组织成模型能读的上下文。

问题思考：

- 为什么同一个问题在新对话和旧对话中可能得到不同回答？
- 如果模型回答“我不知道刚才你说了什么”，可能是能力不足，还是上下文没有被带入？
- system、user、assistant 这些角色标记为什么会影响模型行为？

3 预训练数据与 tokenization

本节导读：

- **本节目的：**说明大语言模型最初从什么材料中学习，以及为什么模型看到的是 token 而不是人类眼中的字符或词。
- **为什么学：**数据过滤决定模型学到什么分布，tokenization 决定模型观察文本的基本单位；二者都会影响模型能力和错误形态。
- **核心内容：**预训练数据来自经过清洗、抽取、过滤、去重和安全处理的大规模文本；tokenization 则把文本压缩成有限词表上的序列。
- **最小例子：**hello world 可能被切成 hello 和 world 两个 token，而中文或罕见词可能被切成更细碎的片段。
- **最该记住：**模型不是直接读“互联网本身”，也不是直接读人类意义上的字词；它读的是被工程流程筛选过、再切成 token 的文本分布。

1. 从互联网到训练语料

- 预训练的第一步通常是收集大规模文本。常见公开数据源包括 Common Crawl。原始网页抓取结果很大，但其中包含广告、导航栏、脚本、重复页面、垃圾站和敏感信息，不能直接拿来训练。

- 数据管道通常会做 URL 过滤、正文抽取、语言过滤、质量过滤、去重和个人可识别信息移除。例如 URL 过滤排除恶意软件站点和低质量来源，正文抽取从 HTML 中去除菜单、页脚和脚本，去重避免模型反复记住同一份材料。
- FineWeb 这类公开数据集可以帮助我们感受量级：最终保留下来的文本可能仍有数十 TB、数万亿 token，但相对于原始互联网已经被强力筛选。
- 因此，训练集本身就是设计产物。哪些语言、网站、文体和领域被保留，会影响模型未来的知识边界、语言能力和偏差。

2. 为什么需要 tokenization

- 神经网络不能直接处理自然语言字符串，它需要有限符号集合上的序列。Tokenization 的作用是把原始文本转换为 token ID，再由 embedding 层映射成向量。
- 最底层可以把文本编码成 UTF-8 字节，每个字节是 0 到 255 的符号。但直接用字节会使序列很长，序列长度又是 Transformer 中非常昂贵的资源。
- 常见方法如 Byte Pair Encoding (BPE) 会反复合并高频相邻片段。这样词表变大，序列变短。常见词、空格加词、标点组合都可能成为单独 token。
- 需要注意，token ID 只是符号编号，不应把它当成有大小关系的普通数字。模型学习的是这些符号在上下文中的统计关系。

3. tokenizer 影响语言能力

- 不同模型的 tokenizer 不同。英文高频片段通常切得较自然，但某些旧 tokenizer 对中文不友好，可能把一个汉字拆成多个 token 或把常见中文词拆得很碎。
- Tokenization 会影响上下文长度和训练效率。同样一段中文，如果被切成更多 token，就会占用更多上下文窗口，也使模型在训练中更难高效学习中文模式。
- 因此，“模型会语言”不等于它天然拥有与人类相同的字符观察方式。它看到的是 tokenizer 定义的符号序列。

4. 由 tokenization 引出的错误

- strawberry 中有几个 r，人类可以直接看字符，但模型可能先看到若干合并后的 token，再从 token 中推断字符结构。这类任务对模型并不天然简单。
- 连续点号计数、拼写操作、每隔几个字符取一个字符、小数位比较等任务，都可能受到 tokenization 和训练分布的干扰。
- 可靠做法是：字符计数、字符串处理和精确算术尽量交给代码工具，不要要求模型只凭语言模式“心算”。

问题思考：

- 为什么“原始互联网很大”不等于“训练数据越多越好”？
- Tokenizer 对不同语言切分效率不同，会如何影响模型的训练成本和使用体验？
- 为什么模型能写流畅文章，却可能数错一个单词中的字母？

4 预训练目标与 Transformer

本节导读:

- **本节目的:** 说明大语言模型训练时到底优化什么目标, 并把这一目标和 Transformer 架构联系起来.
- **为什么学:** 只有理解“预测下一个 token”, 才能理解模型为什么会学到语言、事实、格式和推理模式, 也能理解它的局限.
- **核心内容:** 给定前文 token, 模型输出词表上的概率分布; 训练通过交叉熵提高真实下一个 token 的概率.
- **最小例子:** 在“法国的首都是”这一前缀后, 训练数据中的下一个 token 若对应 Paris, 损失会推动模型提高该 token 的概率.
- **最该记住:** 预训练不是显式灌入事实表, 而是在海量位置上反复做下一个 token 预测.

1. 下一个 token 预测

- 设一段 token 序列为 $x_1, \dots, x_T$. 自回归语言模型的目标是在每个位置根据前文预测下一个 token:

$$p_{\theta}(x_{t+1} \mid x_{\leq t}).$$

其中 θ 表示模型参数, $x_{\leq t}$ 表示当前位置之前的全部上下文.

- 模型通常先输出词表大小的 logits 向量 z_t , 再通过 softmax 得到概率:

$$p_{\theta}(x_{t+1} = v \mid x_{\leq t}) = \frac{\exp(z_{t,v})}{\sum_{u \in \mathcal{V}} \exp(z_{t,u})},$$

其中 $\mathcal{V}$ 是词表.

- 对训练序列最小化负对数似然, 就得到常用的预训练损失:

$$\mathcal{L}(\theta) = - \sum_{t=1}^{T-1} \log p_{\theta}(x_{t+1} \mid x_{\leq t}).$$

这个公式和前面分类模型中的交叉熵是一致的: 真实下一个 token 的概率越低, 损失越大.

2. 为什么简单目标能学到复杂能力

- 为了预测下一个 token, 模型必须学习语法、常识、事实、写作风格、代码格式、数学符号、问答模式和推理痕迹. 这些能力不是逐条规则写进去的, 而是为了降低预测损失被迫形成的内部表示.
- 例如要预测“法国的首都是”后面的 token, 模型需要把“法国”“首都”“Paris”之间的统计关系压缩进参数; 要预测一段代码后面的括号或缩进, 则需要学习程序文本中的结构规律.
- 但这也带来边界: 模型学到的是训练分布中的统计模式, 不是一个完美数据库或定理证明器. 罕见事实、近期事件和精确计算并不天然可靠.

3. Transformer 的计算骨架

- 现代 LLM 多采用 Transformer. 输入 token 先经过 embedding 变成向量, 再反复通过注意力、前馈网络、残差连接和层归一化等模块, 最后输出下一个 token 的 logits.
- 从函数角度看, Transformer 是一个带大量参数的映射

$$F_{\theta} : (x_1, \dots, x_t) \mapsto z_t.$$

它不需要像 RNN 那样严格按时间一步步传递隐状态, 而可以用自注意力让上下文中的位置相互读取信息.

- Transformer 的主要计算是大规模矩阵乘法, 因而非常适合 GPU 并行. 数据中心里的 GPU 集群, 本质上是在更快地处理更多 token、更新更多参数.
- 本讲不展开注意力公式, 因为 W9 会专门讲. 此处只需记住: 自注意力让每个位置能根据上下文动态读取相关信息, 这是 Transformer 能处理长文本依赖的重要基础.

4. 上下文长度与计算成本

- 训练时通常从语料中截取固定长度窗口, 例如几千到更多 token. 窗口越长, 模型能看到的前文越多, 但计算和显存成本也越高.
- 推理时也有上下文长度限制. 如果用户上传的文件、历史对话和工具结果超过窗口, 系统必须截断、摘要或检索其中一部分. 这会影响模型能否准确回答细节问题.
- 因此, “把所有东西都塞进去”不是总是最优策略. 更好的做法是把最相关、最可信、最需要模型直接读取的材料放进窗口.

问题思考:

- 预训练损失为什么可以看作分类交叉熵在词表上的应用?
- 为什么预测下一个 token 会迫使模型学习事实和推理模式, 但又不能保证事实完全可靠?
- 上下文窗口变长时, 对能力和成本分别有什么影响?

5 推理、采样与基础模型

本节导读:

- **本节目的:** 说明训练完成后模型如何生成回答, 并区分基础模型、指令模型和产品中的采样策略.
- **为什么学:** 使用 LLM 时经常遇到同一问题多次回答不同、模型续写跑偏、温度设置影响创造性等现象, 这些都来自推理阶段.
- **核心内容:** 推理时参数通常固定, 系统反复执行“预测概率分布、采样 token、追加上下文”; 基础模型默认续写文档, 指令模型才更像助手.
- **最小例子:** 对同一句 prompt 生成三次, 低温度更可能得到相似答案, 高温度更可能得到发散表达.
- **最该记住:** 推理不是继续训练, 而是在固定参数下逐 token 采样.

1. 推理循环

- 训练结束后, 模型参数通常固定. 用户输入被编码成前缀 token 后, 模型输出下一个 token 的概率分布; 系统选出一个 token, 追加到上下文末尾, 再把更长的上下文送回模型.
- 这个循环一直持续到生成结束标记、达到长度限制或产品层中断. 因而一段回答不是一次性生成的, 而是许多小选择叠出来的.
- 同一个前缀多次运行可能得到不同回答, 是因为采样过程可以带随机性. 这不是模型“变了主意”, 而是概率分布中存在多个可选后续.

2. 温度与采样

- 温度参数 τ 会调整 logits 转成概率时的尖锐程度:

$$p_i^{(\tau)} = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}.$$

当 τ 较小时, 高概率 token 更占优势, 输出更稳定; 当 τ 较大时, 低概率 token 有更多机会被选中, 输出更自由也更容易跑偏.

- 许多产品还会使用 top- p 、top- k 等策略限制候选集合. 这些策略不改变模型参数, 只改变从概率分布中选 token 的方式.
- 因此, 写作、头脑风暴和创意任务可以适当容纳随机性; 事实问答、代码和数学任务则更需要稳定、可核查的输出.

3. GPT-2 作为早期样本

- GPT-2 是理解现代大模型栈的早期样本. 它已经包含生成式预训练 Transformer 的基本形态: 自回归语言建模、Transformer 结构、固定上下文长度和大规模文本训练.
- GPT-2 的参数量约为十几亿, 上下文长度约为 1024 token, 训练数据约为千亿 token 量级. 今天的模型在参数量、上下文长度和训练 token 数上都更大, 但核心抽象仍是预测下一个 token.
- 训练日志中的 loss 下降, 就是在说明模型越来越擅长给真实下一个 token 分配较高概率. 训练初期采样接近乱码, 随着 loss 下降, 文本会逐渐出现局部语法、段落结构和更连贯的内容.

4. 基础模型与指令模型

- 预训练结束后得到的是基础模型 (base model). 它默认做的是续写互联网文档, 而不是扮演助手回答用户. 给它 **What is 2 + 2?**, 它可能回答 **4**, 也可能续写成测验、论坛帖子或教材片段.
- 基础模型仍然很强, 因为它的参数压缩了大量语言和世界知识. 通过 few-shot prompt 给出若干示例, 它可以临时学会翻译、分类、问答等模式, 这称为上下文学习.
- 指令模型 (instruct model) 则经过后训练, 学会把输入解释为用户请求, 并用助手风格回答. ChatGPT 式体验主要来自 instruct model 加产品对话协议, 而不是单纯的 base model.

问题思考:

- 为什么同一个 prompt 在不同温度下会呈现不同稳定性?
- Base model 和 instruct model 在训练目标和默认行为上有什么区别?
- Few-shot prompt 为什么能让基础模型临时完成某个任务?

6 后训练、助手人格与幻觉

本节导读:

- **本节目的:** 说明 ChatGPT 式助手行为如何由后训练塑造, 并解释幻觉为什么是语言模型的系统性风险.
- **为什么学:** 用户真正接触到的大多是后训练模型; 理解 SFT、拒答和事实性训练, 才能知道模型回答中哪些部分可靠、哪些需要核查.
- **核心内容:** SFT 用理想对话样例改变模型的默认续写风格; 幻觉来自模型在缺少事实依据时仍倾向于生成像样答案.
- **最小例子:** 问一个不存在的人名, 模型可能编出作家、运动员或角色介绍; 多次回答身份不同, 就暴露出它没有稳定事实记忆.
- **最该记住:** 流畅、自信、结构完整, 不等于事实正确.

1. 监督微调

- 后训练的第一步常是监督微调 (Supervised Fine-Tuning, SFT). 训练数据从普通互联网文档换成对话样例: 用户提出请求, 理想助手给出回答.
- 算法上, SFT 仍然主要是下一个 token 预测. 区别在于训练材料和需要预测的位置变了: 模型学习在 user 请求之后生成 assistant 应该说的话.
- 标注规范通常会强调有帮助、真实和无害. 这些不是简单写进模型的三条 if-else 规则, 而是通过大量示范改变模型在类似上下文中的生成概率.
- 现代 SFT 数据也会混合人类标注和合成数据. 现有强模型可以生成草稿, 人类再编辑、筛选和补充, 从而扩大后训练数据规模.

2. 助手人格是示例塑造出来的

- 后训练后的模型更像一位受过标注规范训练的助手: 会解释概念、列步骤、道歉、拒绝不合适请求, 也会尽量采用礼貌和结构化表达.
- 这种“人格”不是持续存在的自我, 而是训练分布和当前系统提示共同诱导出的输出风格. 系统消息可以进一步规定回答语气、身份、工具权限和安全边界.
- 因此, 使用模型时应把助手人格看作产品行为层的一部分. 它帮助交互更自然, 但不能把礼貌和自信误认为事实保证.

3. 幻觉的来源

- 幻觉指模型编造事实、人物、来源或细节, 并以自然语言流畅地表达出来. 它不是简单的“故意撒谎”, 而是生成目标和训练分布的副作用.

- 训练集中有大量“谁是某某”“某事发生在何时”这类问题，理想助手通常会自信回答。当用户问到不存在或模型不知道的对象时，模型仍可能复用这种回答模式。
- 另一个原因是参数记忆有损。高频事实更容易稳定记住，罕见事实、训练截止之后的信息和细节引用更容易被模糊模式替代。
- 因此，幻觉最常发生在冷门人名、近期新闻、精确来源、法律医疗金融细节、代码 API 版本和复杂数值结论上。

4. 事实性训练与“不知道”

- 缓解幻觉的一种方法是事实性训练。系统先构造一批有答案的问题，多次让模型回答。如果模型答错或不稳定，就把“我不知道”或“需要查证”的理想回答加入后训练数据。
- 这种方法相当于把模型内部的不确定状态和外部的谨慎表达连接起来。目标不是让模型永不出错，而是让它在证据不足时更愿意承认不确定。
- 用户也可以在 prompt 中要求模型区分确定、推测和需要查证的信息。但更可靠的方法仍是给原始材料、打开搜索、要求引用，或使用代码工具验证计算。

问题思考：

- SFT 为什么能把基础模型从“续写文档”转成“回答用户”？
- 为什么模型越流畅，用户越容易低估幻觉风险？
- 当模型说“不知道”时，这在训练和使用上分别有什么价值？

7 工具使用与工作记忆

本节导读：

- **本节目的：**建立“参数记忆”和“工作记忆”的区别，说明工具调用为什么能显著提升 LLM 的可靠性。
- **为什么学：**现代 GPT 产品不只是模型，还包括搜索、文件读取、代码执行、图像识别等工具；会用工具比单纯追问模型更重要。
- **核心内容：**工具把外部世界的信息或计算结果写回上下文，模型再基于这些结果生成回答。
- **最小例子：**大数乘法不应让模型凭语言模式猜，而应让它调用 Python 得到确定结果，再解释结果。
- **最该记住：**重要事实不要只靠模型背，精确操作不要只靠模型心算。

1. 参数记忆与工作记忆

- 预训练参数像互联网文本的有损压缩包。常见知识可能记得很好，罕见细节可能只有模糊轮廓，近期信息则可能根本不在参数中。
- 上下文窗口像工作台。用户粘贴的原文、上传文件抽取出的文本、搜索结果、代码运行输出和系统提示，都是模型当前可以直接读取的材料。
- 使用 LLM 的关键策略之一，就是把任务所需的可靠材料放到工作台上，让模型基于材料处理，而不是凭模糊记忆补全。

2. 工具调用协议

- 工具调用通常由产品层实现。模型生成某种结构化请求，例如工具名和参数；系统暂停普通文本生成，执行搜索、计算或文件读取；工具结果再作为新上下文返回给模型。
- 对模型来说，工具结果仍然是 token。它并没有神秘地获得浏览器或计算器意识，而是多读了一段由工具产生的文本。
- 模型如何学会何时调用工具，仍然依赖训练样例和产品设计。训练数据要展示查询如何写、参数如何组织、错误如何修正、结果如何引用。

3. 搜索、文件与代码的分工

- 搜索适合近期、变化快、小众、需要引用的问题。例如课程通知、产品功能、比赛时间、新闻背景和股票原因等，都不应只靠模型参数回答。
- 文件上传适合围绕具体材料的任务。论文、课本章节、作业说明、实验数据、报告草稿和代码文件，放进上下文后比让模型凭印象总结可靠得多。
- 代码执行适合精确计算、统计、画图、字符串处理和可复现实验。例如矩阵运算、数值积分、Monte Carlo 模拟、成绩表分析和训练日志可视化，都应优先让程序算。

4. 工具使用的安全边界

- 工具越强，越需要边界。如果做一个本地计算工具，不应直接对模型生成的任意字符串执行 `eval`，因为用户或模型都可能把危险代码带进来。
- 更安全的做法是让工具只接受结构化参数，或用受限解析器检查表达式只包含允许的数字、括号和运算符。对文件系统、网络和命令行工具，更要限制权限并保留日志。
- 工具结果也需要被解释和核验。模型可能选错查询、漏读来源、误解图表，或在自然语言总结时说错自己代码变量的含义。

5. 把生成和检查分开

- 复杂任务中，可以先让模型生成初稿，再让它用工具检查。例如先写实验分析，再运行代码确认数值；先总结论文，再要求列出引用原文中的证据。
- 人类也应参与最后检查。LLM 很像会写代码、会搜索、会整理资料的初级协作者，但它会粗心，也会在不知道时补全。
- 因此，工具使用不是把责任交给模型，而是让模型更容易产生可检查的中间结果。

问题思考：

- 为什么同一篇文章“上传原文后再总结”通常比“直接让模型凭记忆总结”可靠？
- 搜索工具和代码解释器分别解决哪类幻觉或错误？
- 工具调用为什么仍然需要权限控制和人工核验？

8 强化学习与思考模型

本节导读:

- **本节目的:** 说明 SFT 之后为什么还需要强化学习, 以及 thinking model 在数学、代码和多步推理中为什么更有用.
- **为什么学:** 大模型能力提升不只来自更大参数和更多数据, 也来自让模型在可验证任务上自己练习并优化解题策略.
- **核心内容:** SFT 主要模仿示范, RL 让模型采样多条路径并根据奖励强化成功路径; RLHF 用奖励模型近似人类偏好, 但奖励可被过度优化.
- **最小例子:** 同一道数学题采样 20 个解法, 其中 5 个最终答案正确; RL 会提高类似正确路径未来出现的概率.
- **最该记住:** 思考模型不是更有意识, 而是更倾向于展开、检查、回溯和验证.

1. 从看例题到做练习

- 可以把预训练类比为读材料, SFT 类比为看专家例题. 它们让模型知道语言和合格回答大概长什么样, 但不一定找到最适合模型自身的解题策略.
- 强化学习对应做练习题. 给定 prompt, 模型用当前策略 $\pi_{\theta}(y | x)$ 采样多个候选回答 y , 系统用奖励 $r(x, y)$ 判断哪些更好, 再更新模型使高奖励回答更可能出现.
- 一个简化目标可以写成

$$J(\theta) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)}[r(x, y)].$$

这不是要求记住具体优化算法, 而是强调 RL 的目标是提高模型自己生成的候选中的成功概率.

2. 可验证任务中的 RL

- 数学、代码和棋类任务比较适合 RL, 因为它们有相对硬的奖励信号: 最终答案对不对, 单元测试过不过, 对局赢不赢.
- 在这些任务中, 人类不必为每个 prompt 写最佳思路. 模型可以自己尝试多条 token 路径, 系统筛出成功路径. 这些路径可能包含人类没写过、但对模型有效的中间步骤.
- DeepSeek R1 等推理模型展示了这一趋势: 随着 RL 推进, 模型更常生成“重新检查”“换个角度”“验证答案”之类的推理行为. 这些行为是为了提高最终奖励而形成的策略.

3. 思考 token 的作用

- 语言模型每生成一个 token 只有有限计算. 如果要求它第一个 token 就给出复杂题答案, 相当于要求它在一次前向计算中完成全部推理.
- 分步骤写出中间量, 会把这些中间结果放进上下文, 后续 token 可以继续读取. 对模型来说, 逐步推理不仅是展示给人看的解释, 也是把计算展开到 token 序列中的办法.
- 但逐步推理不是万能药. 对精确算术、计数、字符串处理和数值实验, 最可靠的仍是代码工具. 思考模型适合难推理, 工具适合确定性计算.

4. RLHF 的作用与限制

- 许多任务没有硬标准, 例如写诗、写笑话、改语气、总结材料. 对这些不可验证任务, 通常让人类比较候选回答, 再训练 reward model 预测人类偏好, 这就是 RLHF 的核心思想.
- RLHF 的优势是把“人觉得哪个好”转成可优化的信号. 人类从几段回答中排序, 往往比从零写出最佳回答更容易.
- 局限在于 reward model 只是人类偏好的近似. RL 很擅长钻奖励漏洞, 过度优化可能生成在人类看来很差、但奖励模型给高分的回答. 因此 RLHF 通常需要谨慎、有限地使用.

5. 什么时候使用思考模型

- 数学证明、复杂代码调试、多约束规划、算法设计和多步推理, 往往值得使用思考模式.
- 简单事实问答、普通写作润色、短摘要和开放头脑风暴, 通常不必让模型长时间思考. 这会增加等待和 token 成本, 收益却有限.
- 实用策略是先用快速强模型得到初稿; 若问题关键、回答不可靠或推理链很深, 再切换到思考模型或要求模型用工具核验.

问题思考:

- 为什么可自动验证的数学和代码任务特别适合强化学习?
- 思考模型中的“多想一会儿”主要在计算层面意味着什么?
- RLHF 为什么有用, 又为什么不能被当作不可欺骗的真理函数?

9 GPT 应用的任务选择

本节导读:

- **本节目的:** 把 LLM 产品中的模型选择、搜索、深度研究和多模型比较整理成可执行的任务决策框架.
- **为什么学:** 许多失败并不是“模型不行”, 而是用户把近期事实、精确计算、长材料或复杂研究全都当成普通聊天问题处理.
- **核心内容:** 先判断任务是否常见、稳定、低风险; 再决定直接问、搜索、深度研究、上传材料、跑代码或换思考模型.
- **最小例子:** “什么是 Cauchy 列”可以直接问并用教材核对; “今年数学建模竞赛报名截止时间”应搜索官网通知.
- **最该记住:** 会用 LLM 的关键不是把所有问题都丢给聊天框, 而是知道何时换模式和工具.

1. 可以直接问的任务

- 常见、稳定、低风险的问题可以直接问强模型. 例如数学定义回顾、LaTeX 公式写法、英文邮件初稿、课程项目头脑风暴、常见编程概念解释等.
- 直接问并不等于绝对相信. 对课程学习, 最终仍应回到教材、课堂笔记、编译结果或可运行代码核对.
- 适合直接问的问题通常满足三个条件: 训练语料中常见, 知识近期变化不大, 即使初稿有误也容易被用户发现和修正.

2. 必须查证或搜索的任务

- 近期、变化快、小众或需要引用的问题应优先搜索。例如课程调课通知、讲座地点、比赛报名时间、产品最新功能、股票异动原因、新闻事件背景等。
- 搜索不是模型突然拥有实时知识，而是产品把网页摘要或来源文本放进上下文。因而用户仍要检查来源是否权威、查询是否对题、模型是否误读。
- 对高风险问题，如医疗、法律、金融和政策细则，搜索结果也只应作为入口。最终判断要回到权威来源或专业人士。

3. 深度研究适合什么

- Deep Research 可以理解为搜索加长时间阅读和整理。它适合多来源综合、文献入口、方向比较和长报告初稿，不适合只需一个短事实的问题。
- 例如“统计学习和最优化两个方向该怎么选”“某个数学建模题目有哪些资料入口”“某个研究主题有哪些核心论文”这类任务，比普通搜索更适合深度研究。
- Deep Research 的输出像定制研究报告，但仍可能遗漏、误分类或误读来源。引用应被当作阅读入口，不是最终证明。

4. 模型选择与多模型比较

- 同一个产品里可能有大模型、小模型、快速模型、思考模型和不同工具模式。大模型通常更强但更慢更贵；小模型更快更便宜但更容易漏细节。
- 重要任务开始前应看清当前模型和模式：是否有搜索，是否能运行代码，是否是思考模型，是否能读文件或图片。
- 对开放性问题，可以把多个模型当作“委员会”。旅行建议、写作风格、方案比较、产品构思等任务，多模型回答能暴露不同角度。但事实问题仍要回到来源核查。

5. 上下文管理

- 切换话题时新建对话，是最简单的上下文管理。旧任务的长历史会占用窗口，也可能在当前任务中造成干扰。
- 如果任务需要长材料，应主动整理关键部分、上传文件或引用片段，而不是让模型从许多轮闲聊中自己找重点。
- 对复杂项目，可以把上下文分层：当前 `prompt` 写任务目标，附件提供原始材料，自定义指令保留长期偏好，`memory` 只保存稳定个人背景。

问题思考：

- 一个问题是否应该直接问模型，可以从哪些维度判断？
- 搜索结果带引用后，为什么仍然不能完全跳过人工核查？
- 多模型比较适合事实查证，还是更适合开放方案生成？为什么？

10 文件、代码与编程 workflow

本节导读:

- **本节目的:** 说明如何把 LLM 从普通聊天工具变成阅读助手、数据分析助手和代码库协作者.
- **为什么学:** 学习和科研中的许多任务围绕具体材料、数值计算和项目文件展开, 只靠网页聊天容易缺少上下文和可执行验证.
- **核心内容:** 文件上传把材料放进上下文, Python 解释器把精确计算交给程序, 编程代理把模型接入本地代码库.
- **最小例子:** 上传一篇论文后先让模型总结 abstract 和 introduction, 再逐段追问术语和图表; 对论文中的数值实验, 再让 Python 复算或画图.
- **最该记住:** 对具体材料和代码项目, 上下文越贴近原文件, 模型越有用.

1. 文件上传与阅读助手

- 上传 PDF、Word、Markdown、代码或表格, 本质上是把具体材料转成模型可读的上下文. 模型可能“知道”一本经典书, 但不一定精确记得某一章; 上传原文后再问, 更可靠.
- 读论文时, 可以先让模型总结摘要、引言、方法和贡献, 再围绕公式、图表和实验设置逐段追问. 读古典文本或跨领域材料时, 可以让模型解释背景、术语和论证结构.
- 需要注意, PDF 处理可能丢掉图片、表格、脚注或复杂排版. 对关键数字、公式和图表, 应要求模型先转写出来, 用户确认无误后再让它解释.

2. Python 解释器与高级数据分析

- 对精确计算, 模型应写代码并运行, 而不是凭语言模式猜. 大数乘法、矩阵分解、随机模拟、数值积分、拟合曲线和画误差图, 都适合交给 Python.
- 工具调用流程是: 模型写程序, 应用执行程序, 运行结果回到上下文, 模型再解释. 这能把许多模糊回答变成可复现结果.
- 风险在于隐含假设和解释错误. 例如模型可能把缺失值填成 0, 用不合适的曲线外推, 或把图表中的变量解释错. 因此要查看代码、变量和中间输出.

3. Artifacts 与一次性小工具

- 某些产品可以让模型直接生成可运行的浏览器小应用或图示. 例如把一章教材变成 flash-cards, 把论文结构变成 Mermaid 概念图, 把概率论知识点变成交互式练习.
- 这类工具通常没有完整后端或数据库, 更适合个人学习、一次性展示和轻量原型. 它们的价值在于把文本回答变成可操作界面.
- 对视觉学习者, 概念图尤其有用. 它能把核心命题、支撑例子、因果链条和对比关系显式排出来.

4. 真实编程要进入代码库

- 在真实项目中, 不应总在网页聊天里要代码片段再手动复制. 更合适的是使用能读取本地文件、运行命令和跨文件编辑的工具, 例如 Cursor、Windsurf 或 VS Code 插件.

- 这类工具把代码库上下文提供给模型: 当前文件、目录结构、依赖、测试命令和错误日志都能进入上下文. 因此模型能修改多个文件、安装依赖、运行测试并根据反馈修复.
- 这种方式常被称为 `vibe coding`, 但它并不意味着不用懂工程. 模型可能做没看懂的修改、引入未知资源或破坏边界. 用户要审查 `diff`、确认命令、运行测试, 必要时回到传统编程.

5. 学习场景中的具体做法

- 对数学课程, 可以让模型把英文教材段落翻译成中文, 再给出定义、例子和反例; 对证明题, 可以让模型检查逻辑空缺, 但最终证明必须由人确认.
- 对实验课程, 可以让模型读取 `notebook`、运行代码、画训练曲线、解释 `loss` 和 `accuracy`, 再根据错误日志定位问题.
- 对论文阅读, 可以先让模型整理符号表, 再把关键公式转成可计算小例子. 如果公式涉及数值, 应用 `Python` 验证.

问题思考:

- 为什么上传原始文件后, 模型仍可能误读图表或公式?
- 用 LLM 做数据分析时, 哪些中间结果最应该让用户检查?
- 网页聊天代码片段和代码库代理在上下文上有什么根本区别?

11 多模态、记忆与自定义工具

本节导读:

- **本节目的:** 说明 LLM 应用如何从文本扩展到语音、图片、视频和长期个性化, 并解释自定义 GPT 为什么可以看作保存下来的 `prompt` 工具.
- **为什么学:** 现代 LLM 产品能力变化很快, 但多数新功能仍可用 `token`、上下文、工具和产品层协议来理解.
- **核心内容:** 语音、图像和视频可以被转写、编码或抽帧后进入上下文; `memory` 和 `custom instructions` 则把长期偏好预先放入上下文.
- **最小例子:** 拍一张板书照片, 先让模型 OCR 成 `LaTeX`, 用户检查后再让它解释推导步骤.
- **最该记住:** 多模态越强, 越要确认模型到底看见了什么、听见了什么、记住了什么.

1. 语音输入与原生音频

- 最常见的语音功能其实是“假音频”: `speech-to-text` 先把用户声音转成文字, LLM 仍然读文本; `text-to-speech` 再把模型文字读出来. 它足够实用, 能显著减少打字.
- 原生音频能力则让模型直接处理音频 `token`, 能理解语气、实时对话、改变声音风格或被打断. 这类能力在产品上更自然, 但背后仍可理解为音频被编码成模型可处理的序列.
- 使用时要注意专有名词、代码名、变量名和数学符号容易转写错. 重要输入说完后应检查文字转写.

2. 图片输入、图片生成与视频

- 图片输入可以用于读板书、课件截图、教材公式、论文图表、营养标签、仪器读数和界面错误。关键做法是先让模型转写或描述，用户确认后再追问解释。
- 图片生成通常由产品把用户 prompt 或对话摘要交给图像模型完成。它适合海报、封面、图标和创意素材，但生成图像中的文字、数字和细节仍要核对。
- 视频理解可以看作图片理解的延伸：产品可能连续处理视频帧，也可能抽取关键帧。从用户角度像是在“看视频”，从模型角度仍是把视觉信息转成可处理表示。

3. 记忆与自定义指令

- 当前对话上下文是短期工作记忆，New Chat 会清空它。Memory 则是跨对话保存的用户资料摘要，例如用户正在学什么、偏好什么解释风格、长期项目背景是什么。
- Custom instructions 是用户主动写的全局行为设定。例如“解释数学概念时先给直觉，再给定义，最后给一个可计算例子”，或“回答不要写成营销文案”。
- 二者本质上都是把长期信息预先放到新对话上下文里。它们能提升个性化，也带来隐私和偏见问题，因而需要定期管理和删除不需要的记忆。

4. 自定义 GPT 与可复用 prompt

- Custom GPT 通常不是训练了一个新模型，而是把 instructions、示例、文件、工具权限和输出格式保存成一个可复用小工具。
- 它适合高频重复任务。例如英文数学术语卡片生成器、论文段落翻译器、板书 OCR 到 LaTeX 工具、错题拆解助手、代码审查清单和固定格式摘要器。
- 设计自定义 GPT 时，不只要描述任务，还要给少量输入输出示例。Few-shot prompt 能帮助模型理解格式边界、颗粒度和判断标准。
- 对长示例，可以使用清晰的标签分隔输入和理想输出。这和教学生类似：只讲规则不如给例子，给例子还要让边界清楚。

5. 最终使用原则

- 多模态、记忆和自定义工具扩展了 LLM 的输入输出形式，但没有取消基本原则：重要事实查来源，精确计算跑代码，具体材料放上下文，高风险结论找权威。
- 模型输出应被当作可用草稿、阅读助手、资料整理器、代码协作者和工具调度器，而不是最终权威。
- 用户真正需要掌握的能力，是把问题拆成合适的 LLM 工作流：直接问、搜索、上传、运行、画图、读图、思考、比较、核验。

问题思考：

- 为什么图片和视频输入时，“先转写再解释”比直接听模型解释更稳妥？
- Memory 和 custom instructions 都会影响新对话，它们分别适合保存什么信息？
- 什么时候应该把一个 prompt 固化成 custom GPT 或类似工具？

12 小结

- 大语言模型的核心计算是根据上下文预测下一个 token. 预训练让模型学习语言、知识和常见推理模式; 后训练让它以助手方式回答.
- Base model 默认续写文档, instruct model 才更像助手. ChatGPT 式体验还依赖系统提示、对话模板、采样策略和产品工具.
- 幻觉来自模糊参数记忆和生成目标的共同作用. 流畅输出不等于事实正确, 尤其是近期、冷门、精确和高风险信息.
- 工具使用的本质是把搜索结果、文件内容、代码输出和多模态识别结果放进上下文, 让模型基于工作记忆处理任务.
- 强化学习让模型在可验证任务上练习自己的解题策略; RLHF 则用奖励模型近似人类偏好, 但不能被当作不可欺骗的标准.
- 实际使用 LLM 时, 应按任务选择模式: 稳定低风险问题可直接问, 近期事实用搜索, 长材料上传文件, 精确计算跑代码, 复杂推理用思考模型, 重复工作固化成自定义工具.

问题思考:

- 如果要用 LLM 辅助完成一份课程论文, 哪些步骤适合直接让模型做, 哪些步骤必须由人核查?
- 当模型回答看起来合理但没有来源时, 你会怎样设计下一轮 prompt 或工具调用来验证它?
- 学完本讲后, 你会如何向没有技术背景的人解释“ChatGPT 为什么有用, 但不能盲信”?